

第E1课：PyTorch实验

深度学习体系结构

PyTorch介绍

- 1. 定义与背景

- 开源机器学习库
- 由Facebook AI Research (FAIR) 开发
- 广泛用于计算机视觉、自然语言处理等深度学习领域

- 2. 核心特性

- 动态计算图：运行时构建，灵活调试
- 张量 (Tensor)：基本数据结构，支持GPU加速
- 自动微分 (autograd)：自动计算梯度，简化反向传播

- 3. 关键模块

- **torch.nn**：神经网络层与损失函数
- **torch.optim**：优化算法，如SGD、Adam
- **torch.utils.data**：数据加载与预处理

- 4. 分布式训练

- 支持多GPU和多节点并行训练
- 加速模型训练过程

- 5. 生态系统

- 丰富的第三方库和工具
- 如torchvision (计算机视觉)、torchtext (自然语言处理)

- 6. 社区支持

- 庞大社区，包括开发者、研究人员
- 提供教程、文档、示例代码及活跃论坛

PyTorch安装

- **查看电脑是否支持CUDA**：打开NVIDIA控制面板，查看支持的CUDA版本。
- **下载并安装CUDA**：访问NVIDIA官网下载适合自己电脑支持的CUDA版本，并按照提示进行安装。
- **下载并安装Anaconda**：访问Anaconda官网下载安装包，并按照提示进行安装。
- **创建虚拟环境**：在Anaconda Prompt中创建新的虚拟环境，例如命名为PyTest。
- **下载并安装Pytorch**：根据选择的CUDA版本，从PyTorch官网获取安装命令，并在Anaconda Prompt中执行该命令。

安装Anaconda

- 下载Anaconda

- 访问Anaconda的官方网站（[Anaconda官网](#)）或清华大学开源软件镜像站（[清华大学镜像站](#)），下载最新版本的Anaconda安装包。

- 安装Anaconda

- 运行下载的安装包，按照提示完成安装。在安装过程中，可以选择将Anaconda添加到系统路径中，以便在命令行中直接使用conda命令。

- 验证安装

- 打开命令行窗口（在Windows上可以是CMD或PowerShell，在Linux或macOS上可以是Terminal），输入conda --version，如果显示Anaconda的版本号，则说明安装成功。

创建Python环境

- 使用Anaconda，你可以为PyTorch创建一个单独的Python环境，这有助于隔离项目依赖项并避免冲突。
- **打开Anaconda Prompt：**
 - 在Windows上，你可以通过开始菜单找到并打开Anaconda Prompt。
- **创建新环境：**
 - 输入以下命令来创建一个新的Python环境（以Python 3.8为例）：
`conda create -n pytorch_env python=3.8`
 - 根据提示输入y以确认创建环境。
- **激活环境：**
 - 输入以下命令来激活你刚刚创建的环境：
`conda activate pytorch_env`

安装PyTorch

- **选择安装版本：**

- 根据你的系统配置（CPU或GPU）和CUDA版本（如果使用GPU），选择合适的PyTorch版本。你可以在[PyTorch官网](#)找到详细的安装指南和版本信息。

- **使用Conda安装PyTorch：**

- 如果你已经激活了Python环境，并且知道要安装的PyTorch和CUDA版本，你可以使用以下命令进行安装（以PyTorch 1.12.1、CUDA 11.3为例）：

```
conda install pytorch==1.12.1 torchvision==0.13.1  
torchaudio==0.12.1 cudatoolkit=11.3 -c pytorch
```

- 这条命令会从PyTorch的官方conda频道安装指定版本的PyTorch、TorchVision、TorchAudio和CUDA Toolkit。

- **验证安装：**

- 安装完成后，你可以通过以下命令来验证PyTorch是否已成功安装：

```
python -c "import torch; print(torch.__version__);  
print(torch.cuda.is_available())"
```

- 如果PyTorch安装成功，并且你的系统支持CUDA（如果你安装了CUDA版本），则第二条打印语句应该返回True。

验证安装

- 在命令行中输入以下命令来验证PyTorch是否安装成功：
- ```
python import torch print(torch.__version__)
print(torch.cuda.is_available())
```
- 如果没有报错，并且`torch.cuda.is_available()`返回`True`，则表示PyTorch已经成功安装，并且可以使用GPU。
- 请按照上述步骤进行操作，如果遇到任何问题，可以进一步查找相关信息或寻求帮助。

# 注意事项

- **显卡驱动：**

- 如果你打算使用GPU版本的PyTorch，请确保你的NVIDIA显卡驱动是最新的，或者至少是与你要安装的CUDA版本兼容的。

- **CUDA版本：**

- CUDA版本的选择取决于你的显卡型号和驱动程序版本。在选择CUDA版本时，请务必查看NVIDIA的官方文档以获取兼容性信息。

- **Anaconda环境管理：**

- 使用Anaconda环境管理可以帮助你更好地控制项目依赖项和Python版本。
- 如果你不使用Anaconda，你也可以使用pip来安装PyTorch，但请注意管理依赖项可能会更加复杂。



# 问题解决

- 在安装PyTorch时遇到错误，可以尝试以下几种解决方案：
- **切换国内源**：更换国内源可以显著提高conda安装速度，避免因网络问题导致安装失败。你可以添加清华、阿里云或上海交大的镜像源到channels列表中，并更新conda。
- **使用Anaconda自带pip安装**：更换国内源后，可以使用Anaconda自带的pip安装PyTorch。
- **确保Python版本符合要求**：PyTorch对Python版本有一定要求，目前Windows下支持Python 3.7到3.9。如果你的Python版本不满足要求，请升级到支持的版本。
- **检查CUDA版本**：如果你计划使用GPU进行计算，可能会遇到与你的NVIDIA CUDA版本不匹配的问题。请确保你安装了与你的GPU和CUDA版本兼容的PyTorch版本。你可以在PyTorch官方网站上查找支持的CUDA版本。
- **解决依赖包冲突**：在安装PyTorch时，可能会与其他Python包发生冲突。尝试在虚拟环境中安装PyTorch，这样可以确保PyTorch与其他包互不干扰。
- **更新pip和setuptools**：如果出现“未能找到wasi”的错误，可以尝试更新你的pip和setuptools到最新版本。使用以下命令进行更新：  
`pip install --upgrade pip setuptools`
- 然后重新尝试安装PyTorch。
- **操作系统兼容性问题**：在某些操作系统上安装PyTorch时可能遇到兼容性问题。例如，Windows上较旧的Visual C++版本可能导致问题。尝试在更新的操作系统上运行安装程序，或者手动更新你的Visual C++版本以解决兼容性问题。你还可以考虑使用Docker镜像来运行PyTorch，以避免操作系统兼容性问题。
- **使用国内镜像加速安装**：在国内访问官方的PyTorch下载链接可能会遇到速度慢或无法访问的问题。可以使用国内的镜像源来安装PyTorch，例如阿里云、上海交大和清华的镜像源。

# YOLO V5安装

# 安装YOLO V5所需环境

- **安装Python和pip**
  - 确保你的计算机上已安装Python 3和pip。Python可以从[Python官网](#)下载并安装，pip通常随Python一起安装。
- **安装Anaconda ( 可选但推荐 )**
  - Anaconda是一个流行的Python数据科学平台，它简化了包管理和环境配置。你可以从[Anaconda官网](#)下载并安装。
  - 安装完成后，你可以使用conda命令来创建和管理虚拟环境。
- **创建虚拟环境**
  - 使用conda或virtualenv创建一个新的虚拟环境。这有助于隔离项目依赖，避免与其他项目发生冲突。
    - 使用conda创建环境：conda create -n yolov5env python=3.8 ( 确保使用与YOLO v5兼容的Python版本 )
    - 激活环境：conda activate yolov5env
  - 如果你选择不使用Anaconda，也可以使用virtualenv来创建虚拟环境。
- **安装PyTorch**
  - YOLO v5基于PyTorch框架，因此你需要安装PyTorch。
  - 访问[PyTorch官网](#)，根据你的操作系统和CUDA版本选择合适的安装命令。
  - 如果你有NVIDIA GPU并希望利用CUDA加速，确保已安装适当版本的CUDA和cuDNN，并按照PyTorch官网的指示进行安装。

# 安装YOLO v5

- 下载YOLO v5源码

- 访问YOLO v5的GitHub仓库，下载最新的源码。

<https://github.com/ultralytics/yolov5>

- 将下载的源码解压到一个合适的目录。

```
git clone https://github.com/ultralytics/yolov5 # clone
```

- 安装依赖

- 进入YOLO v5的源码目录，你会看到一个名为requirements.txt的文件，它列出了项目所需的所有依赖包。

- 使用pip安装这些依赖：pip install -r requirements.txt

```
cd yolov5
```

```
pip install -r requirements.txt # install
```

- 如果你在中国大陆，可能会遇到网络问题导致安装失败。此时，你可以考虑使用国内的镜像源，如阿里云、清华大学开源软件镜像站等。

- 验证安装

- 安装完成后，你可以尝试运行YOLO v5提供的一些示例脚本来验证安装是否成功。
- 通常，这些脚本会加载预训练的权重文件，并对图像或视频进行目标检测。

# 注意事项

- **版本兼容性**
  - 确保你安装的Python、PyTorch、CUDA和cuDNN版本与YOLO v5兼容。
  - 在安装过程中，如果遇到版本不兼容的问题，你可能需要调整这些组件的版本。
- **网络问题**
  - 如果你在中国大陆使用pip安装依赖时遇到网络问题，可以尝试使用国内的镜像源来加速下载。
- **GPU加速**
  - 如果你希望利用GPU加速目标检测，请确保你的计算机上已安装NVIDIA GPU，并正确配置了CUDA和cuDNN。
- **虚拟环境**
  - 强烈建议在虚拟环境中安装YOLO v5，以避免与其他项目产生依赖冲突。

# YOLO v5 推论代码

- import torch
- from PIL import Image
- import cv2
- import numpy as np
- 
- **# Load the YOLOv5 model**
- model =  
torch.hub.load('ultralytics/yolov5',  
'yolov5s', pretrained=True)
- 
- **# Load an image**
- img\_path =  
'path/to/your/image.jpg' # Replace  
with your image path
- img = Image.open(img\_path)
- 
- **# Perform inference**
- results = model(img)
- 
- **# Print results**
- results.print() # Print results to  
console
- 
- **# Show results**
- results.show() # Display results in a  
window
- 
- **# Save results**
- results.save(save\_dir='inference/output') # Save results to 'inference/output'  
directory
- 
- **# Get the results as a pandas  
DataFrame**
- df = results.pandas().xyxy[0] # xyxy  
format
- print(df)

# 训练YOLO模型

# 步骤 1: 设置环境

- **克隆YOLOv5仓库:**

- 打开您的终端 ( Terminal ) 或命令提示符 ( Command Prompt ) , 然后输入以下命令来克隆YOLOv5的GitHub仓库 :

```
git clone https://github.com/ultralytics/yolov5.git
cd yolov5
```

- 这将下载YOLOv5的代码库并将其放置在您当前的工作目录中。

- **安装依赖项:**

- 确保您已经安装了Python和pip ( Python的包管理工具 ) 。然后 , 在YOLOv5的仓库目录中运行以下命令来安装所需的依赖项 :

```
pip install -r requirements.txt
```

- 这将安装YOLOv5运行所需的所有Python库。



## 步骤 2: 准备数据集

- **组织数据集:**  
将您的数据集组织成以下格式：
  - images文件夹：包含所有图像文件。
  - labels文件夹：包含与images文件夹中图像相对应的标签文件（.txt格式）。
- 每个标签文件应包含以下格式的行：class\_id x\_center y\_center width height，其中坐标是归一化的（值在0和1之间）。
- **创建数据集YAML文件:**
  - 创建一个YAML文件（例如custom\_dataset.yaml），并指定您的数据集路径和类名。YAML文件的内容可能如下所示：

```
train: ../datasets/custom_dataset/images/train # 训练集图像路径
val: ../datasets/custom_dataset/images/val # 验证集图像路径
nc: 2 # 类别数
names: ['class1', 'class2'] # 类别名称
```

请根据您的数据集实际路径和类别名称进行相应修改。

## 步骤 3: 配置模型

- 选择模型配置:
  - YOLOv5提供了多种模型配置（例如 `yolov5s.yaml`, `yolov5m.yaml`等），这些配置位于`models`目录中。
  - 您可以根据需要修改这些配置，例如更改类别数以匹配您的数据集。
  - 通常，您只需要修改`nc`（类别数）和`names`（类别名称）字段。

## 步骤 4: 训练模型

- 运行训练脚本：  
使用train.py脚本来开始训练：

```
python train.py --img 640 --batch 16 --epochs 100 --data
path/to/custom_dataset.yaml --cfg models/yolov5s.yaml --weights
yolov5s.pt --name custom_training_run
```

- 请根据您的实际需求替换以下参数：
  - img：图像尺寸（例如640）。
  - batch：批处理大小（例如16）。
  - epochs：训练轮数（例如100）。
  - data：数据集YAML文件的路径。
  - cfg：模型配置文件。
  - weights：开始训练时使用的预训练权重（例如yolov5s.pt）。
  - name：训练运行的名称（用于保存结果）。

## 步骤 5: 评估和使用模型

- **评估模型:**

- 训练完成后，使用验证数据集评估模型性能。
- YOLOv5的训练脚本通常会在每个epoch结束时自动评估模型。

- **推理:**

- 使用训练好的模型对新图像或视频进行推理。使用 detect.py 脚本：

```
python detect.py --weights
runs/train/custom_training_run/weights/best.pt --
source path/to/images_or_videos
```

- 请根据您的实际需求替换以下参数：
  - --weights：训练好的模型权重文件路径。
  - --source：要检测的图像或视频路径。

# YOLO Dataset YAML

- **YAML文件的作用**

- YOLO数据集的YAML文件主要用于定义数据集的训练集、验证集和测试集的文件路径，以及数据集中的类别数量和类别名称。

- **YAML文件的示例**

```
data.yaml
train: ./datasets/images/train # 训练集图像文件目录
val: ./datasets/images/val # 验证集图像文件目录
test: ./datasets/images/test # 测试集图像文件目录（可选）
nc: 3 # 类别数量
names: ['dog', 'cat', 'car'] # 类别名称列表
```

## 注意事项

- **数据增强:** YOLOv5默认包含数据增强技术，这有助于提高模型性能。
- **超参数调优:** 尝试不同的超参数以优化模型性能。
- **硬件:** 强烈建议使用GPU进行训练，以加快训练速度。



# GPT4All

- 是一个开源软件生态系统，它允许任何人在日常硬件上训练和部署强大且定制的大型语言模型（LLM）。
- **1. GPT4All开源软件生态系统**
  - **后端**：负责管理和执行大型语言模型的推理。
  - **绑定**：将C API绑定到Python等高级编程语言。
  - **API**：提供REST API接口，获取生成结果和嵌入向量。
  - **聊天客户端**：跨平台聊天应用程序，用户体验友好。
- **2. 指令调优**
  - GPT4All采用指令调优方法。
  - 用户可定制模型，实现多样化任务。
- **3. 支持的模型与架构**
  - 支持的架构：LLaMa、GPT-J、MPT。
  - 量化处理模型，降低运行资源需求。
  - 用户可贡献后端代码，添加新架构支持。
  - 提供一系列预训练模型，选择多样。



# GPT4All安装

- 下载GPT4All安装包
  - 访问GPT4All的官方网站：GPT4All官网 (<https://www.nomic.ai/gpt4all>)。
  - 在网站上找到与自己操作系统（Windows、Linux 或 MacOS）匹配的安装包，并点击下载。
- 安装GPT4All
  - Windows系统：
    - 双击下载的安装包（如gpt4all-installer-win64.exe），按照安装程序的提示进行安装。
    - 在安装过程中，可能需要选择安装路径和是否添加GPT4All到系统路径。

# 安装前置依赖

- GPT4All的运行依赖于一些前置软件包和Python库。
- 在安装GPT4All之前，需要确保这些依赖项已经安装。
- 安装Python：
  - GPT4All需要Python环境来运行。可以从Python官方网站下载并安装最新版本的Python。
- 安装所需的Python库：
  - 打开命令行或终端，导航到GPT4All的安装目录。
  - 使用pip命令安装GPT4All所需的所有Python库。例如，可以运行`pip install -r requirements.txt`（如果GPT4All提供了requirements.txt文件）。

# 运行GPT4All

- 从GPT4All的官方网站或其他可靠来源下载预训练的GPT模型文件。
- 将下载的模型文件放置到GPT4All指定的目录中。通常，这个目录是GPT4All安装目录下的某个子目录（如chat或models）。
- 运行GPT4All
  - 打开命令行或终端，导航到GPT4All的安装目录。
  - 运行GPT4All的启动脚本或命令。例如，可以运行 `python gpt4all.py`（如果GPT4All提供了这样的启动脚本）。
  - GPT4All启动后，将加载指定的GPT模型，并提供一个文本交互界面供用户使用。

# GPT4All的Python客户端

- 如果GPT4All提供了一个专门的Python客户端库，你可以直接安装该库并使用它提供的函数来与模型进行交互。
- 这通常涉及到使用pip安装客户端库，然后导入并使用它。
- **pip install gpt4all-client** # 假设GPT4All有一个名为gpt4all-client的Python客户端库